Web Architecture & Protocols

Week 5 · Foundations module · Textbook Chapter 5

Brian C. Keegan, Ph.D.
Associate Professor, Department of Information Science
University of Colorado Boulder

September 14, 16 & 18, 2026

This week at a glance

We open the hood on the web — the layered stack of protocols that turns a URL into a rendered page, and turns web data collection from *magic* into *engineering*.

Monday | Concepts & notebook intro

- Client–server, TCP/IP, DNS, HTTP, and URLs as one stack

Wednesday | Notebook lab

- Trace the stack in Python: DNS lookups, HTTP diagnosis, a live API call turned into a time series

Friday | Textbook revisions

- Propose & peer-review pull requests improving Chapter 5

Companion notebook

`ch-05-protocols.ipynb`

Bring a laptop

Wednesday is hands-on — we watch the requests fly.

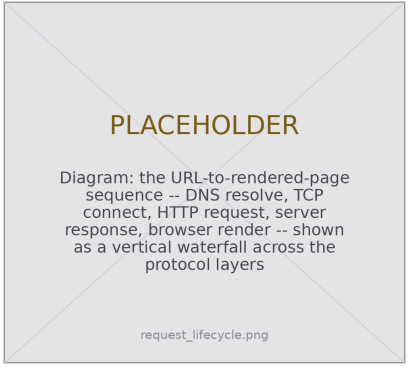
1. Monday · Concepts

What happens when you load a web page?

Type a URL, press Enter, and in milliseconds a precise sequence unfolds:

1. **DNS** resolves the domain name to an IP address
2. **TCP/IP** opens a connection to that server
3. **HTTP** sends a request formatted to spec
4. the server returns a **response**
5. the browser **renders** the HTML, CSS, and JavaScript

Understanding this sequence transforms web data collection **from magic into engineering —** and turns debugging from guesswork into systematic diagnosis.



URL to rendered page, one layer at a time.

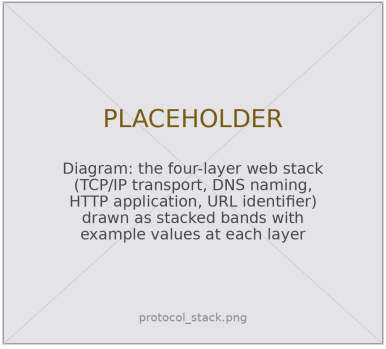
Textbook Ch. 5, "What Happens When You Load a Web Page?"

The lower stack: client–server, TCP/IP & DNS

Every scrape and API call is one **request–response** cycle: a **client** (your browser, your Python script) asks a **server** for a resource.

TCP/IP — the transport. Every machine on a public network has an IP address. IPv4 offers $2^{32} \approx 4.3$ billion of them; IPv6 offers $2^{128} \approx 3.4 \times 10^{38}$. Data hops router to router to arrive.

DNS — the address book. It maps a name like `en.wikipedia.org` to an IP. Names are hierarchical — `subdomain.domain.TLD` — and services follow conventions: `api.agency.gov`, `data.agency.gov`.



Four protocols, one layered conversation.

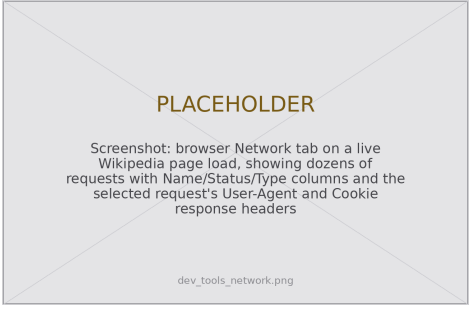
Textbook Ch. 5, “The Client–Server Model,” “TCP/IP,” and “DNS.”

See it yourself: browser developer tools

Before writing a line of code, inspect the page by hand. Open dev tools with `Ctrl+Shift+I` or `F12` (`Cmd+Opt+I` on a Mac).

Inspector shows the HTML as a tree you can hover and expand. Start on a simple Wikipedia article — commercial sites bury structure under tracking scripts.

Network reveals every request a page makes — often hundreds: HTML, CSS, JS, images, fonts, trackers. Each shows a URL, method, status code, and headers like `User-Agent` and `Cookie`.



The Network tab: one page load, dozens of requests.

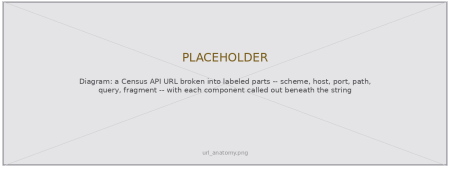
Textbook Ch. 5, "Browser Developer Tools."

The upper stack: HTTP status codes & URLs

HTTP structures the conversation: a request carries a method (GET, POST, …) and headers; a response carries a **status code**:

- 200 OK — the body has your data
- 301/302 redirect — follow `Location`
- 403 forbidden — often a missing `User-Agent`
- 404 not found 429 slow down
- 500 the server's problem, not yours

URLs identify the target, with a fixed structure:



`scheme://host:port/path?query#fragment`

Textbook Ch. 5, "HTTP: The Application Layer" & "URLs: Anatomy and Parsing."

What the notebook covers

This week's companion notebook walks *down* the stack, then back *up*:

- **Addressing** — find your local and public IP (`socket`, `ipify`); resolve a domain with `dns.resolver`; trace the route to Wikipedia
- **Diagnosis** — a `diagnose_request()` helper that reads status codes, headers, and redirect history
- **A live API** — the Wikimedia pageviews endpoint: fix a 403, parse the JSON, and plot a time series
- **Scaling up** — a `Session` with retries; parsing *and* building URLs with `urllib.parse`

Connecting to Ch. 2 (Ethics) & Ch. 3 (Post-API)

"View source" and network inspection exist because the web was built as **open** public infrastructure — open standards in public RFCs, not proprietary products. That same openness is what platforms increasingly fight, the tension you will study in the post-API age.

2. Wednesday · Notebook Lab

Where am I? Where is the server?

Your public IP is how the internet sees you; DNS is how a name becomes a routable address:

```
import requests
import dns.resolver

# Your public IP, as servers on the internet see it
print(requests.get("https://api.ipify.org?format=json").json())
# {'ip': '128.138.xxx.xxx'}

# Resolve a domain name to its IP address(es)
for rec in dns.resolver.resolve("en.wikipedia.org", "A"):
    print("IP address:", rec)
```

`scapy` traceroute needs root

The notebook's `traceroute("en.wikipedia.org")` maps each router hop, but raw packets require admin privileges (Npcap on Windows). Can't run it? Read along — nothing later depends on it.

Textbook Ch. 5, "TCP/IP" & "DNS." `socket.gethostbyname` often returns a loopback address — query an external service for your real public IP.

Diagnosing HTTP errors, not guessing

The status code says *what* failed; the full response says *why*. Read both:

```
def diagnose_request(url, headers=None):
    try:
        r = requests.get(url, headers=headers, timeout=10)
    except requests.exceptions.ConnectionError:
        return print("Connection failed:", url)
    print("Status:", r.status_code)
    print("Content-Type:", r.headers.get("Content-Type", "n/a"))
    print("Bytes:", len(r.content))
    if r.history: # a redirect chain
        print("Redirected from:", r.history[0].url)
    return r
```

A **403** from an API usually means a missing `User-Agent` (a one-line fix); a **403** from a news site may block scraping outright. A **429** means slow down; a **500** is the server's problem — retry later.

Textbook Ch. 5, "Diagnosing HTTP Errors."

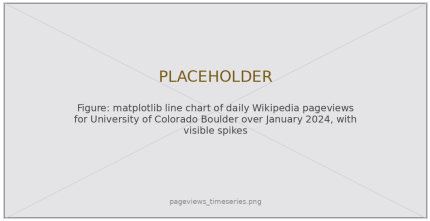
Make an API call, get a time series

The Wikimedia pageviews API rejects the default client with **403** — add a meaningful `User-Agent` and it returns **200**:

```
url = ("https://wikimedia.org/api/rest_v1/"
       "metrics/pageviews/per-article/en.wikipedia/"
       "all-access/all-agents/"
       "University_of_Colorado_Boulder/daily/"
       "20240101/20240131")
ua = {"User-Agent": "WebDataScience/1.0 (you@colorado.edu)"}
data = requests.get(url, headers=ua).json()
```

Reshape the nested JSON, then plot it:

```
df = pd.DataFrame(data["items"])
df["timestamp"] = pd.to_datetime(df["timestamp"],
                                 format="%Y%m%d00")
df.plot(x="timestamp", y="views")
```



Daily pageviews — spikes line up with real-world events.

Textbook Ch. 5, "Making API Requests with Custom Headers."

A loop over one host is your cue for a Session

A bare `requests.get()` opens a fresh connection every time. Looping over many URLs at one host? Create a `Session` — it reuses the connection, remembers headers, and retries automatically:

```
from requests.adapters import HTTPAdapter
from urllib3.util.retry import Retry

session = requests.Session()
session.headers.update({"User-Agent": "WebDataScience/1.0 (you@colorado.edu)"})

# Retry 429/5xx with backoff of 1s, 2s, 4s; honors Retry-After
retries = Retry(total=3, backoff_factor=1,
                status_forcelist=[429, 500, 502, 503, 504])
session.mount("https://", HTTPAdapter(max_retries=retries))

for article in ["Web_scraping", "Data_science"]:
    r = session.get(f"https://en.wikipedia.org/wiki/{article}")
```

Six lines of setup replace the hand-rolled backoff loop — and carry over unchanged into the scraping, Wikipedia, and government-data chapters.

Textbook Ch. 5, “Connections, Sessions, and Retries.”

Parse — and build — URLs with the right tool

Never parse a URL with a regular expression. `urllib.parse` follows the spec:

```
from urllib.parse import urlparse, parse_qs, quote

u = urlparse("https://api.census.gov/data/2022/acs/acs5?get=NAME&for=place:07850")
u.scheme, u.netloc, u.path # 'https', 'api.census.gov', '/data/2022/acs/acs5'
parse_qs(u.query) # {'get': ['NAME'], 'for': ['place:07850']}
```

Two API styles need two builders. **Path-segment** APIs need `quote()` on each component; **query-parameter** APIs take a `params=` dict and let `requests` do the escaping:

```
quote("AC/DC", safe="") # 'AC%2FDC' -- the slash MUST be encoded
requests.get(base, params={"titles": "University of Colorado Boulder"})
```

Mixing them up — query-encoding a value the API expects in its path — is a classic source of confusing 404s.

Textbook Ch. 5, "URLs: Anatomy and Parsing" & "Building URLs Programmatically."

Lab: work these, then show-and-tell

Work through the chapter's exercises in pairs:

1. Network tab: a Wikipedia article vs. a news site — how many requests? What fraction is content vs. tracking?
2. Resolve five related domains with `dns.resolver` — do any share an IP address?
3. Plot pageviews for three related articles; hypothesize what caused the spikes
4. Write `diagnose_url()` and test it on a 200, a 404, a redirect, an API, and a blocked site
5. **5617:** profile a site's delivery infrastructure — redirect chain, CDN headers, traceroute hops — in a 500-word memo

Show-and-Tell

Come ready to share:

- a challenging bug
- interesting data you pulled
- a provocation for a research design

When a request fails, don't guess.
Ask the stack, in order:

Can I reach the server?
Am I resolving the right address?
What status code came back?
Are my headers correct?
Is my URL well-formed?

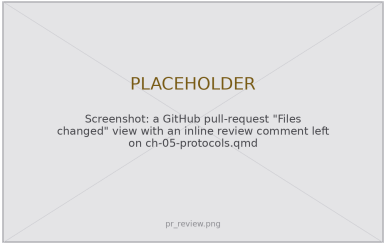
3. Friday · Textbook Revisions

Improving Chapter 5 together

Today we make the book better. Each of you opens a **pull request** against `ch-05-protocols.qmd` and reviews a classmate's.

The workflow (from Week 1):

1. Branch / fork the book repo
2. Edit the chapter; commit with a clear message
3. Open a PR describing *what* and *why*
4. Review two peers' PRs: kind, specific, actionable



Contributions & reviews count toward the Textbook Revisions grade (30%).

A revision menu for Chapter 5

High-value targets — pick one and scope it to a single PR:

- **Test the code against live services.** `api.ipify.org`, `ip-api.com` (45 req/min), the Wikimedia pageviews API, and DNS results all drift. Run the notebook, report what broke, and fix it.
- **Deepen "Common Issues to Debug."** Add a worked `Retry-After` example, an IPv6 `AAAA` lookup, or a step-by-step `scapy`/Npcap install note — the chapter's thinnest section.
- **Add a figure.** A clean layered-stack diagram, an annotated Network-tab screenshot, or a labeled URL-anatomy figure would anchor the concepts.
- **Strengthen an exercise.** Give the `diagnose_url()` exercise expected output, a rubric, or a starter cell so it is self-checking.
- **Extend "Further Reading."** Tie the HTTP/1.1 RFC or MDN's HTTP overview to the "openness" callout on the web as public infrastructure.

Targets drawn from the chapter's callouts, "Common Issues to Debug," and "Further Reading."

What makes a good revision & a good review

A strong PR

- One focused change, clearly titled
- Explains the problem it fixes
- Builds (`quarto render`) without errors
- Matches the book's voice and notation
- Discloses any AI assistance

A strong review

- Runs / reads the change, not just skims
- Names specifics (line, claim, code)
- Separates "must fix" from "nice to have"
- Is generous — assume good faith
- Ends with a clear verdict

Next week: **Parsing static web pages** — turning messy HTML into a clean DataFrame.

Key takeaways

1. The web is a **layered stack**: TCP/IP transports, DNS resolves names, HTTP structures the conversation, URLs identify the target.
2. Debugging is **systematic diagnosis**, not guesswork: reach the server? right address? which status code? headers correct? URL well-formed?
3. A meaningful `User-Agent` fixes most **403**s; backoff or a `Retry` policy handles **429**s.
4. A `for` loop over one host is your cue to use a `Session` — persistent connection, shared headers, automatic retries.
5. **Never regex a URL.** `urllib.parse` both parses and builds; know path-segment (`quote`) from query-parameter (`params=`) APIs.

Reading: Textbook Ch. 5. Related: Mitchell (2018); Vanden Broucke et al. (2018); Mozilla Developer Network (2023).